Architecture

In this section, a global overview of the Jitsi infrastructure is provided. If you just started contributing to the project, we highly recommend reading this section thoroughly.

Components

Jitsi comprises a collection of projects:

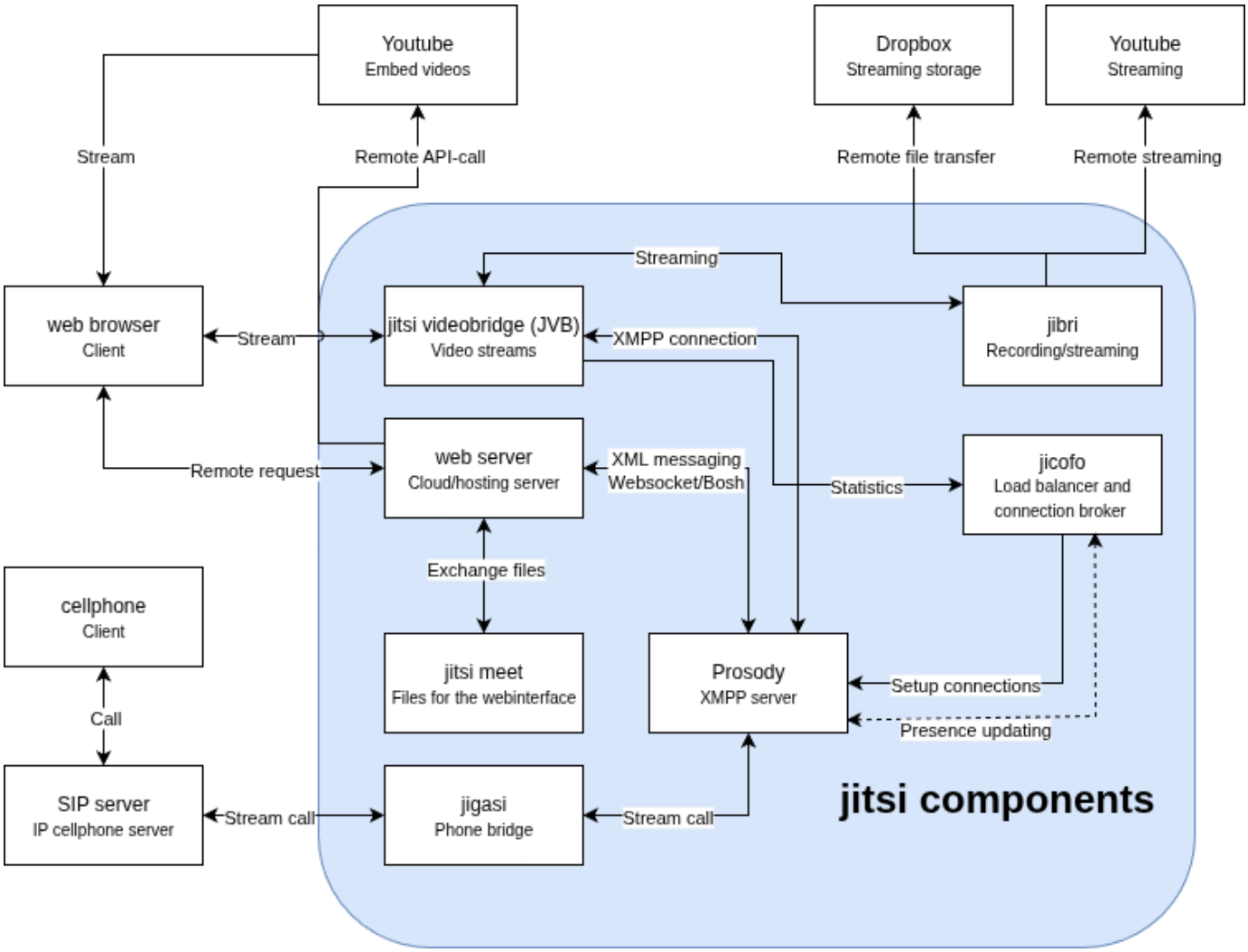
- Jitsi Meet - WebRTC compatible JavaScript application that uses Jitsi Videobridge to provide high-quality, scalable video conferences. Build upon React and React Native.
- Jitsi Videobridge (JVB) - WebRTC compatible server designed to route video streams amongst participants in a conference.
- Jitsi Conference Focus (jicofo) - server-side focus component used in Jitsi Meet conferences that manages media sessions and acts as a load balancer between each of the participants and the videobridge.
- Jitsi Gateway to SIP (jigasi) - server-side application that allows regular SIP clients to join Jitsi Meet conferences
- Jitsi Broadcasting Infrastructure (jibri) - set of tools for recording and/or streaming a Jitsi Meet conference that works by launching a Chrome instance rendered in a virtual framebuffer and capturing and encoding the output with ffmpeg.

External Software used by Jitsi:

- Prosody - XMPP server used for signalling

Architecture Diagram

The individual connections between the previously described components, as well as their external integrations are described in the figure below.



The external connections can be categorized into two main groups. Firstly, the connections between clients that request a video or audio connection are performed through remote requests and data streams. The second category of external connections is those to external services that help store recordings, stream recordings, stream videos, or help with creating meetings.

Code Map

In this section, we will look at the main parts of the codebase and see what they can be used for.

./react/features This folder is where it is best to start writing your code, as it contains most of the app components that are used in the apps on Android and iOS, as well as on the web version. This source folder is split up into all the different features that Jitsi has to offer, such as authentication, chat interaction, keyboard shortcuts, screenshot capture, remote control, and virtual background. Each of these features has a folder in this map, which is then again split up to keep a hierarchy and consistency throughout the code.

Each feature folder consists of a subfolder called components, in this folder all of the React, or React Native for mobile, components are expressed. Usually, in this folder there will be a separation between native and web components, however, in some cases, the same component could be used for both Android, iOS, and web browsers, in which case there is no separation made.

As stated before, the codebase mostly consists of React and React Native, which is the React version for mobile applications. Most of the features make use of the so-called class component by React [^class-comp], however, some new features start to use the new way to write functional components by using hooks[^func-comp].

The application makes use of React Redux as well, this is used as a general state store to keep track of important parameters that are used throughout the application. More on React Redux can be found here [^react-redux].

Most features also contain a file called `middleware.js`. This file acts as a bridge between the component and the functionality of the rest of the application.

./modules/external-api In this folder, the external API can be found. This API can be used in various events like participants joining/leaving the meeting, changes in avatars or chat, as well as errors in using the microphone or camera.

./android and ./ios Both of these folders contain the basics of the Android and iOS apps respectively. However, the code for the application itself and its components can be found in the **react/features** folder, which is explained earlier in this section.

./conference.js This file can be found at the root of the project and contains the foundation of any interaction between a user and a conference room. This consists of setting up a connection to it, joining the meeting room, muting and unmuting, but also functions to gather information about the participants that are in the room.

./lang This folder contains all the different translations that are present in Jitsi Meet. The translations can be found in the code with each of the keys in the translation maps that can be found in the `main-[language].json` files.

./css This folder contains all the CSS that is used in the project. The files (mostly .scss files[^scss]) are split up into features like the React features that they are used in.

Testing

The main form of testing code changes is done through torture tests, next to this the code is tested manually.

The torture tests are located in a separate repository, Jitsi Meet Torture. The project contains end-to-end tests for several key functions such as peer-to-peer and invites. The testing can be done for iOS, Android, and the web, which are all the platforms that Jitsi Meet can be used on. The testing is done automatically for pull requests by project members, where it is used in combination with the continuous integration by a Jenkins instance running the tests, testing on the meet.jit.si instance. Other members can run the tests locally. The test results can be viewed on an automatically generated web page.

Manual testing is performed while doing code reviews, however, there are also testing releases that can be freely downloaded and deployed or can be used on the beta test server.

Edit this page

*Last updated on **May 28, 2026***